AUTOMATED VEHICLE CLASSIFICATION & COUNTING

AVC&C | Group G25 | University of Bradford

SYSTEM DESIGN DOCUMENT

CDIO Design Phase | Architecture | Pipeline | Data Flow | Interface Design

1. Design Overview

This document describes the system architecture and detailed design of the AVC&C system. The Design phase (Sprints 3–4) translated Bradford Council's requirements into a concrete technical architecture, data flow specification, and user interface design.

The system was designed around three non-negotiable constraints: CPU-only hardware deployment, full GDPR compliance with no personal data capture, and a user interface accessible to non-technical Bradford Council traffic planning officers.

2. System Architecture

2.1 High-Level Pipeline

The AVC&C system follows a five-layer linear processing pipeline. Figure 1 below shows all layers from camera input to dashboard output.

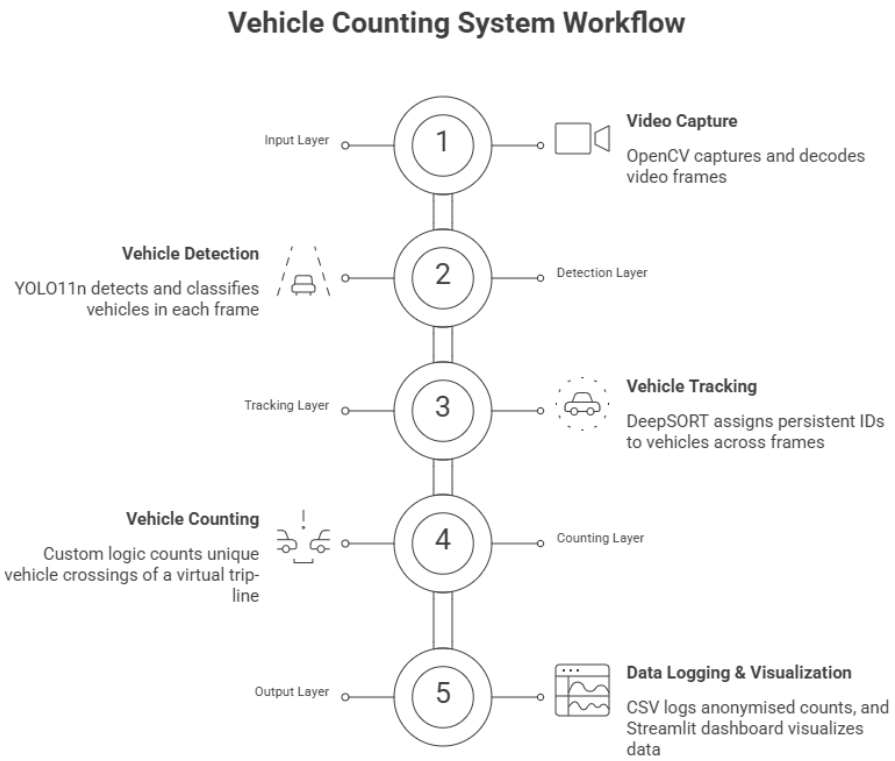

Figure 1 — AVC&C System Pipeline (5 layers: Input → Detection → Tracking → Counting → Output)

Layer	Component	Technology	Responsibility
1	Input Layer	OpenCV VideoCapture — DroidCam / webcam / video file	Capture and decode video frames at target FPS
2	Detection Layer	YOLO11n (avc_model_v15/weights/best.pt)	Detect and classify vehicles per frame
3	Tracking Layer	DeepSORT — Kalman filter + deep appearance	Persistent track IDs across frames
4	Counting Layer	Custom centroid + virtual trip-line logic	Count unique vehicle crossings
5	Output Layer	CSV logger + Streamlit dashboard (app.py)	Store anonymised counts + visualise

2.2 Component Interaction

Figure 2 shows how all system components interact — from camera input through to Bradford Council officers viewing the dashboard.

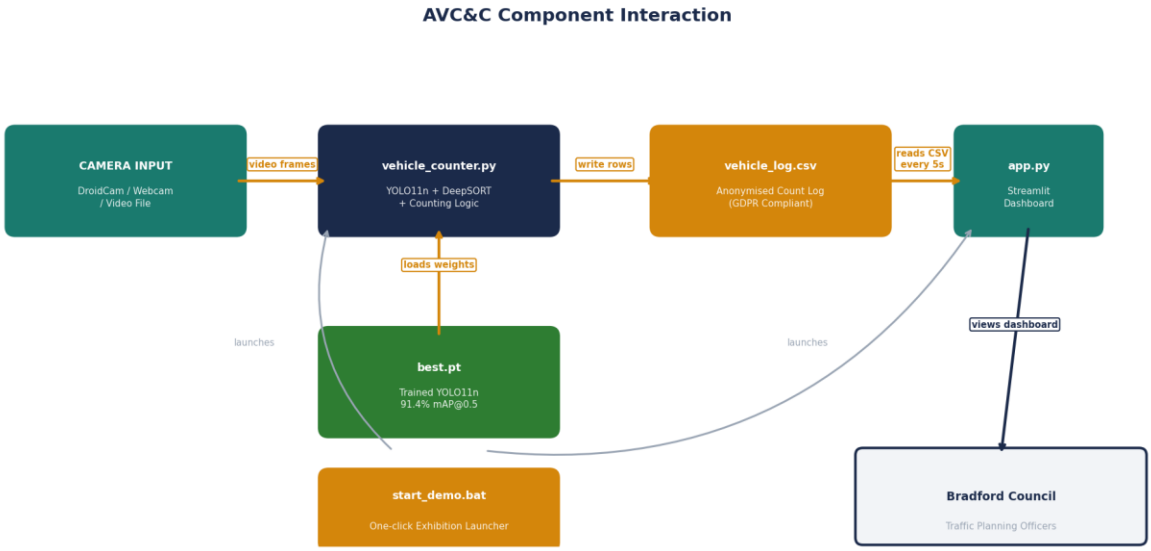

Figure 2 — Component Interaction Diagram (vehicle_counter.py, app.py, best.pt, vehicle_log.csv, start_demo.bat)

2.3 Data Flow

Figure 3 shows the complete step-by-step data flow from video frame capture to dashboard display, with GDPR-compliant data handling at every stage.

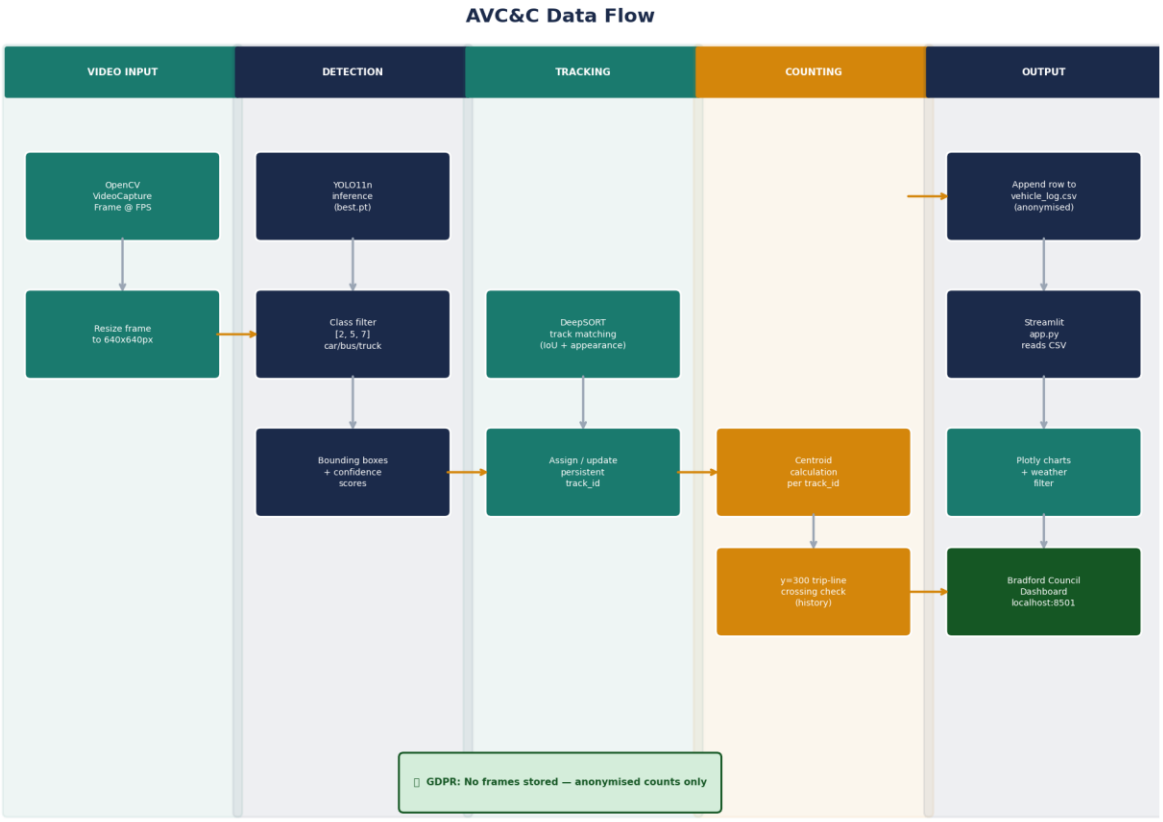

Figure 3 — AVC&C Data Flow (7 steps from frame capture to Streamlit dashboard, GDPR compliant throughout)

3. Component Design

3.1 vehicle_counter.py — Core Detection & Counting Script

The primary system script. Designed by Ayush Acharya during Sprints 3–4.

Design Decision	Rationale
COCO class filter [2, 5, 7]	Restricts to car/bus/truck only. Eliminates false positives from pedestrians and other COCO classes.
Trip-line at y=300 (fixed)	Mid-frame position captures vehicles after full entry. Avoids edge artefacts. Configurable in v2.0.
Centroid history per track_id	Solves double-counting bug. Crossing only confirmed on directional change, not proximity to line.
Set for counted IDs	Python set() for O(1) lookup — prevents duplicate counting even if track re-enters frame.
Colour-coded bounding boxes	Green = car, Blue = bus, Orange = truck. Enables quick visual verification during demo and testing.
CSV append mode	Log written in append mode — not overwrite. Long-running collection without data loss on restart.
Weather CLI argument	Weather condition passed at runtime. Allows Bradford Council to tag data without code modification.

3.2 app.py — Streamlit Dashboard

The client-facing dashboard. Designed and built by Aman Gill during Sprint 6.

Design Decision	Rationale
st.rerun() with 5s sleep loop	Live dashboard without WebSocket complexity. 5-second interval balances freshness vs CPU load.
Reads from vehicle_log.csv	Decoupled from vehicle_counter.py — dashboard and counter run as independent processes.
Weather condition dropdown	Enables Bradford Council planners to isolate traffic patterns by condition — direct client requirement.
Plotly bar chart — counts by class	Communicates vehicle type volumes. Colour-coded to match bounding box scheme.
Plotly line chart — counts over time	Enables trend analysis and peak traffic identification using timestamp field.

3.3 train.py — Training Pipeline

Custom YOLO11n training script. Designed by Ayush Acharya for Sprint 5.

Parameter	Design Rationale
Base: yolo11n.pt (COCO pretrained)	Transfer learning — cars/buses/trucks already in pretraining distribution. Faster convergence.
Epochs: 50	Sufficient for convergence on 11,582 images without overfitting. Monitored via val loss curve.
Batch: 8	Maximises CPU RAM utilisation without memory errors on AMD Ryzen 5 5600H (16GB RAM).
imgsz: 640	Standard YOLO inference size. 1280px tested — too slow on CPU (below 15 FPS threshold).
Output: avc_model_v15/best.pt	best.pt saves highest val mAP checkpoint. Validated at 91.4% mAP@0.5 — used for all inference.

4. Dataset Design

4.1 prepare_dataset.py

Automated script to organise the 11,582-image Roboflow export into YOLO training directory structure.

Split	Images	Percentage	Purpose

Train	7,337	70%	Model weight updates during backpropagation
Validation	2,532	22%	Overfitting monitor — evaluated but weights not updated
Test	1,713	15%	Held-out final evaluation — never seen during training
TOTAL	**11,582**	**100%**	5 weather conditions equally distributed across all splits

5. File Structure

Figure 4 shows the complete GitHub repository file structure with each file's purpose and category.

Figure 4 — GitHub Repository File Structure (colour-coded by file type and purpose)

File / Directory	Purpose
vehicle_counter.py	Core detection, tracking, counting, and CSV logging
app.py	Streamlit dashboard — reads vehicle_log.csv and renders charts
train.py / avc_model_v2.py	Custom YOLO11n training pipeline
prepare_dataset.py	Automated dataset split organisation for YOLO training
start_demo.bat	One-click exhibition launcher (Windows batch file)

vehicle_log.csv	Output: anonymised count log — timestamp, class, confidence, x, y, weather
runs/detect/avc_model_v15/weights/best.pt	Trained YOLO11n — 91.4% mAP@0.5
data.yaml	YOLO training configuration — class names, dataset paths
README.md	GitHub documentation — setup, usage, architecture overview
requirements.txt	Python dependency specification for reproducible setup

6. Interface Design

6.1 Dashboard Layout (app.py)

UI Element	Design Choice	User Benefit
Summary metrics header	Large bold numbers for total count, last update	Immediate status at a glance
Weather filter dropdown	Always visible at top of page	Quick filter without scrolling
Bar chart — vehicle types	Green/blue/orange matching bounding boxes	Consistent visual language across system
Line chart — counts over time	X = timestamp, Y = cumulative count	Trend and peak traffic identification
Auto-refresh indicator	'Last updated: HH:MM:SS' in header	Officers know data freshness

7. Design Constraints & Decisions

7.1 CPU-Only Constraint

- **YOLO11n nano selected:** Smallest variant — fastest CPU inference at acceptable accuracy
- **Single camera input:** Multi-camera rejected — too slow on CPU without GPU parallelism
- **batch=8 for training:** Maximum before RAM exhaustion on 16GB laptops
- **640px inference:** 1280px tested — dropped below 15 FPS threshold on CPU

7.2 GDPR Constraint

- **No frame storage:** Video discarded immediately after inference — never written to disk
- **COCO class filter as enforcement:** Model never attempts to detect faces or plates by design
- **Anonymised CSV only:** track_id is session-local integer — not linked to vehicle identity across sessions

7.3 Usability Constraint

- **Streamlit over Flask/Django:** No HTML/CSS/JS knowledge required to use or maintain
- **start_demo.bat:** Removes command-line requirement entirely for day-to-day use
- **Weather dropdown (not text input):** Enforces consistent weather labels — prevents data entry errors

8. Design Validation

Checkpoint	Validation Activity	Outcome
Sprint 2 Review	Dr Panesar reviewed YOLO11n + DeepSORT + Streamlit stack	**Approved to proceed**
Sprint 2 Client Check	Architecture summary confirmed with Yunus Mayat	**Client confirmed requirements met**
Sprint 4 Retrospective	Double-counting bug fixed — centroid history approach	**48/48 manual count match**
Sprint 6 Review	Dashboard demonstrated to Dr Panesar	**Praised in sprint blog entry**